

Backend Architecture Review

Context: Professional SaaS Backend (Javalin / PostgreSQL) **Goal:** Enterprise-grade security, scalability, and multi-tenancy.

- **Security (CSRF Vulnerability):** You are extracting auth from a cookie (`ctx.cookie("zen_session")`) and your CORS config sets `it.allowCredentials = true`. This combination makes the app vulnerable to Cross-Site Request Forgery (CSRF), where a malicious site could trick a logged-in user's browser into making a request to your API.
 - *Action:* How will you mitigate this? Will you implement an Anti-CSRF token header, or ensure your cookie creation strictly enforces `SameSite=Strict` (or `Lax`)?
- **Architecture (Missing Tenant Context):** In the `.before` `bouncer`, you verify the token and extract the user: `String userEmail = JwtUtil.verifyAndGetEmail(token);`. However, our database schema is a multi-tenant SaaS relying on `tenant_id`. If a user belongs to Zen 63, the backend needs to know that.
 - *Action:* Can we update `JwtUtil` to also extract the `tenant_id` and role from the JWT payload, and add them via `ctx.attribute("tenantId", tenantId)` so your route handlers know exactly which business data to query?
- **Maintainability (Hardcoded Origins):** In your CORS setup, origins like `"https://zen-63.vercel.app"` and `"http://localhost:63342"` are hardcoded. As you white-label this or spin up staging environments, recompiling to add domains will become a bottleneck.
 - *Action:* How about we extract these allowed origins into your `.env` file and load them dynamically through your `DatabaseConfig.loadEnvironment()` method?

